

中国科学院大学

University of Chinese Academy of Sciences

Generative Models & AI4S Applications

Lecture 7 Normalizing Flow I

Fall 2026

Liu Yunfan (刘云帆)

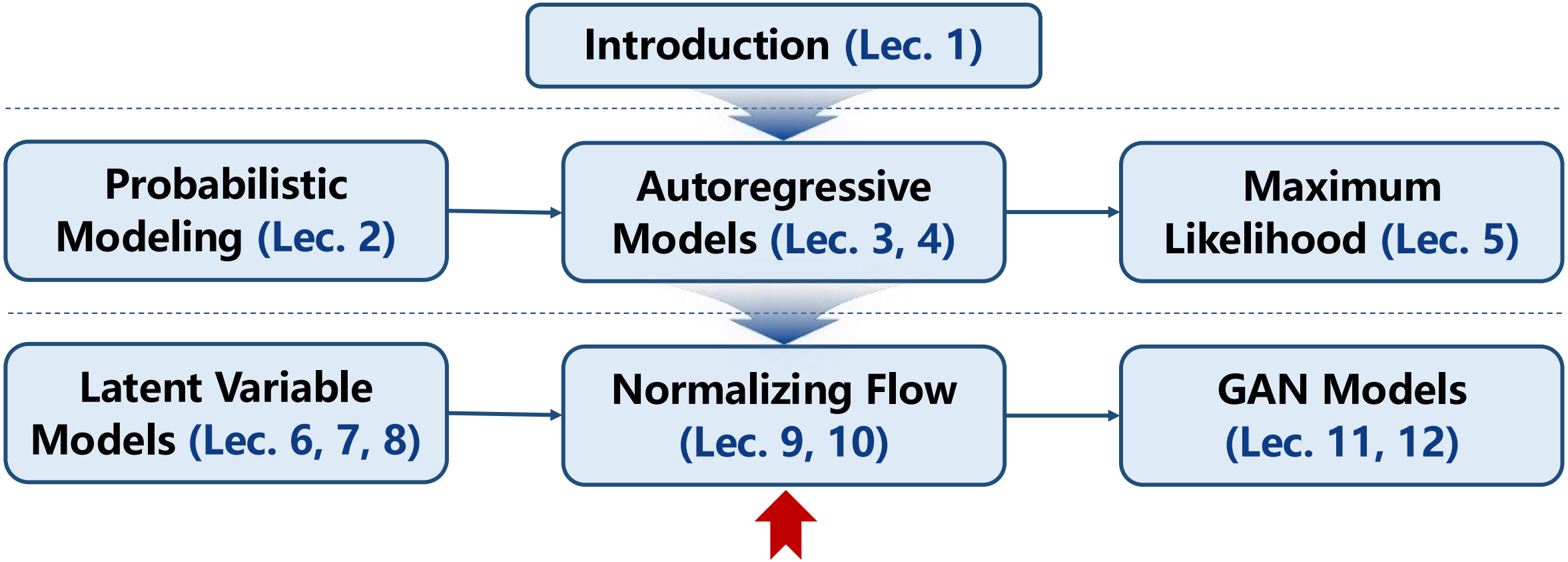
liuyunfan@ucas.ac.cn

Announcements

- Assignment 1 is due on **October 18th**

- Assignment 2 will be released after the deadline of assignment 1

Roadmap



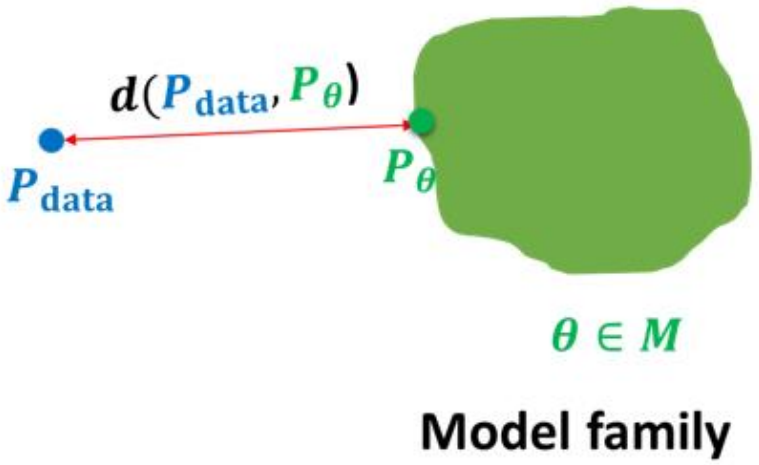
Introduction to various generative models

Recap: Learning a Generative Model

Dog Images



$x_i \sim P_{\text{data}}$
$i = 1, 2, \ldots, n$



Dog Image Generator

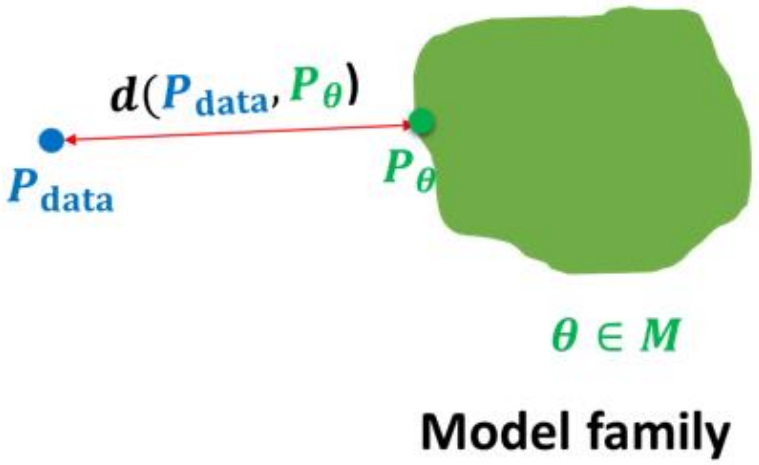
- **Goal:** learn a probability distribution $p_\theta(x)$ over data points x
- Machine learning methods:
  - **Model Family**
  - **Learning Objective**
  - **Optimization**
- Functions of generative models:
  - **Generation**
  - **Density Estimation**
  - **Representation Learning**

Recap: Learning a Generative Model

Dog Images



$x_i \sim P_{\text{data}}$
$i = 1, 2, \ldots, n$



Dog Image Generator

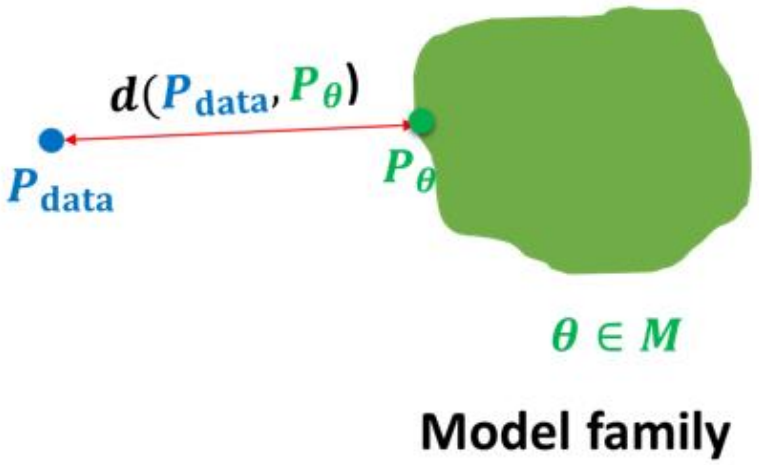
- **Questions**: ① How to represent $p_\theta(x)$? ② How to optimize θ?
- Machine learning methods:
  - **Model Family**
  - **Learning Objective**
  - **Optimization**
- Functions of generative models:
  - **Generation**
  - **Density Estimation**
  - **Representation Learning**

Recap: Learning a Generative Model

Dog Images



$x_i \sim P_{\text{data}}$
$i = 1, 2, \ldots, n$



Dog Image Generator

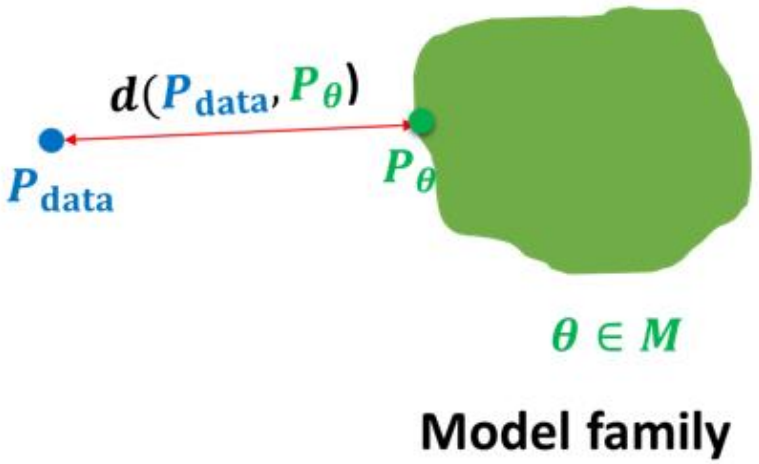
- **Questions**: ① How to represent $p_\theta(x)$? **② How to optimize θ?**
- Machine learning methods:
  - **Model Family**
  - **Learning Objective** ⇒ **Maximum Likelihood**
  - **Optimization**
- Functions of generative models:
  - **Generation**
  - **Density Estimation**
  - **Representation Learning**

Recap: Learning a Generative Model

Dog Images



$x_i \sim P_{\text{data}}$
$i = 1, 2, \ldots, n$



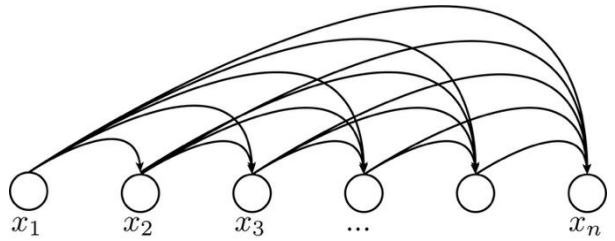
Dog Image Generator

- **Questions**: ① **How to represent $p_\theta(x)$?** ② How to optimize θ?
- Machine learning methods:
  - **Model Family**
  - **Learning Objective**
  - **Optimization**
- Functions of generative models:
  - **Generation**
  - **Density Estimation (AR)**
  - **Representation Learning (VAE)**

Recap: How to represent $p_\theta(x)$?

- **Autoregressive Models:**

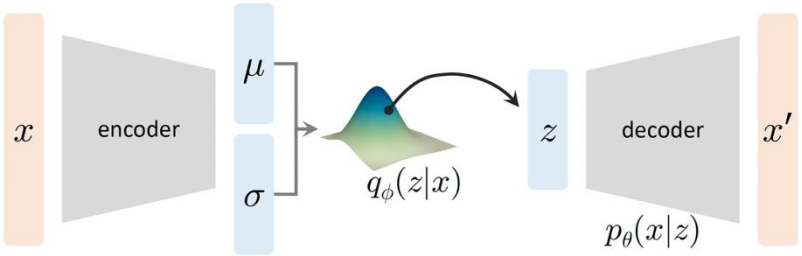
$$p_\theta(x) = \prod_{i=1}^{n} p_\theta(x_i \mid x_{<i})$$



  - ➢ **Pros:** Compute tractable likelihoods
  - ➢ **Cons:** No direct mechanism for representation learning

- **Latent Variable Models:**

$$p_\theta(x) = \int p_\theta(x, z)\, dz$$



  - ➢ **Pros:** Naturally performs representation learning via latent variable z
  - ➢ **Cons:** Marginal likelihoods are intractable

Can we design a **latent variable model** with **tractable likelihoods**?

Lecture Outline

- Introduction to Flow-based Models
- Review: Change of Variable Formula
- Normalizing Flow Models
- Discussion and Advanced Topics

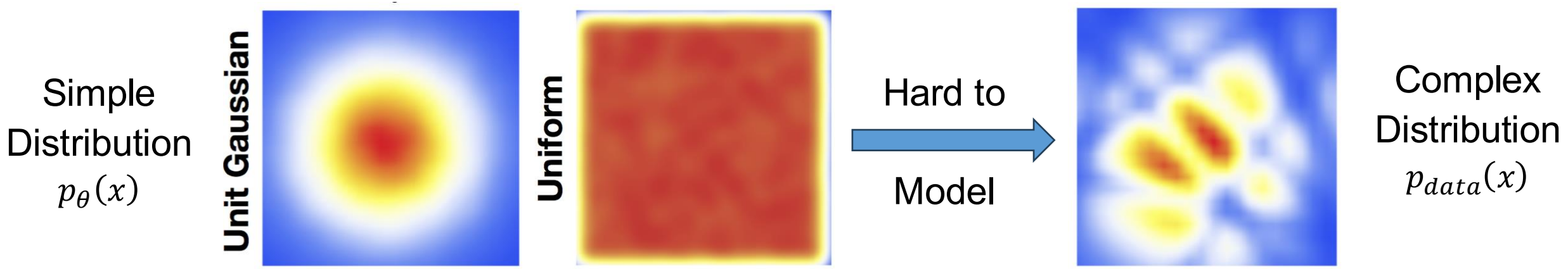


Lecture Outline

- **Introduction to Flow-based Models**
- **Review: Change of Variable Formula**
- **Normalizing Flow Models**
- **Discussion and Advanced Topics**

Introduction to Flow-based Models

- **From Simple Prior to Complex Data Distributions**
  - **Goal:** A model distribution $p_\theta(x)$ with desired properties
    - ✓ Training: Easy-to-evaluate likelihood (e.g., closed-form density)
    - ✓ Generation Easy-to-sample
  - **Note:** Simple distributions satisfy the above requirements

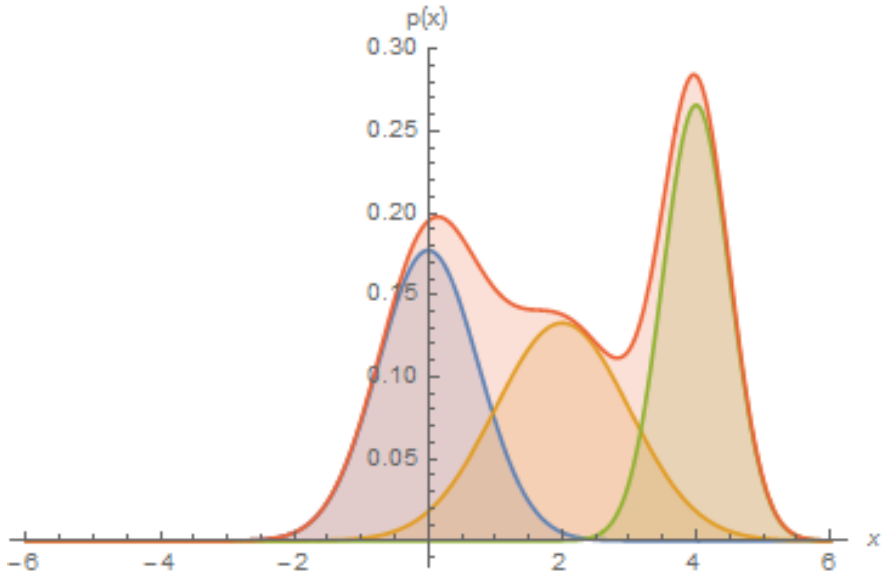


➔ Unfortunately, they are **incapable of representing complex p_{data}**

Introduction to Flow-based Models

- **From Simple Prior to Complex Data Distributions**
  - ➢ **Motivation:** **Transforming** simple distributions to represent complex ones
    - ✓ **Latent Variable Models** is one kind of these models

Gaussian Mixture Models (GMM)

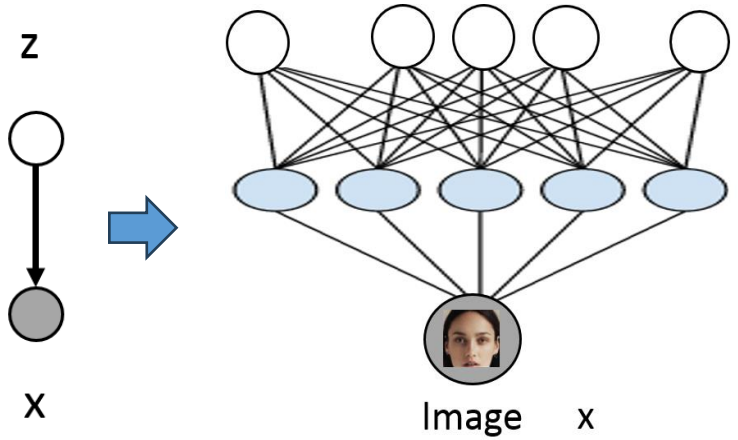


$$p(\mathbf{x}) = \sum_{\mathbf{z}} p(\mathbf{x}, \mathbf{z}) = \sum_{\mathbf{z}} p(\mathbf{z}) p(\mathbf{x} \mid \mathbf{z}) = \sum_{k=1}^{K} p(\mathbf{z} = k) \underbrace{\mathcal{N}(\mathbf{x}; \mu_k, \Sigma_k)}_{\text{component}}$$

Introduction to Flow-based Models

- **From Simple Prior to Complex Data Distributions**
  - ➢ **Motivation:** **Transforming** simple distributions to represent complex ones
    - ✓ **Latent Variable Models** is one kind of these models

Variational Autoencoder (VAE)



① Simple prior: $z \sim p(z) = \mathcal{N}(0, I)$

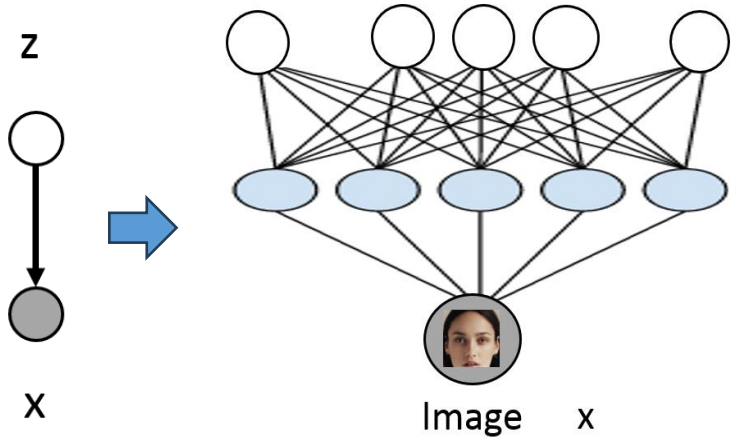
② Transform via $p_\theta(x \mid z) = \mathcal{N}\left(\mu_\theta(z), \Sigma_\theta(z)\right)$

➔ Both $p(z)$ and $p_\theta(x \mid z)$ are simple, but $p_\theta(x)$ can be complex

Introduction to Flow-based Models

- **From Simple Prior to Complex Data Distributions**
  - ➢ **Motivation:** **Transforming** simple distributions to represent complex ones
    - ✓ **Latent Variable Models** is one kind of these models

Variational Autoencoder (VAE)



What are the issues of latent variable models?

① Simple prior: $z \sim p(z) = \mathcal{N}(0, I)$

② Transform via $p_\theta(x \mid z) = \mathcal{N}\left(\mu_\theta(z), \Sigma_\theta(z)\right)$

➔ Both $p(z)$ and $p_\theta(x \mid z)$ are simple, but $p_\theta(x)$ can be complex

Introduction to Flow-based Models

- **From Simple Prior to Complex Data Distributions**
  - **Motivation:** Transforming simple distributions to represent complex ones
    - ✓ **Latent Variable Models** is one kind of these models
  - **Question:**
    - ✓ Do latent variable models have the desired properties for $p_\theta(x)$?
      - ① **Training:** Easy-to-evaluate likelihood
      - ② **Generation:** Easy-to-sample
    - ✓ No! The likelihood $p_\theta(x) = \int p_\theta(x \mid z)\, p(z)\, dz$ is **intractable**
      - ➔ Needs to enumerate all z (no one-on-one mapping)
  - **Solution:**

    Create a **deterministic** and **invertible** mapping f_θ between x and z



Lecture Outline

- Introduction to Flow-based Models
- Review: Change of Variable Formula (变量变换公式)
- Normalizing Flow Models
- Discussion and Advanced Topics

Review: Change of Variable Formula

- **Change of Variables in 1-D: A Toy Example**
  - Let Z be a uniform random variable $\mathcal{U}[0, 2]$ with density p_Z
    - ✓ What is $p_Z(1)$?
  - Let $X = 4Z$, what is $p_X(4)$?
    - ✓ $p_X(4) = P(X = 4) = P(4Z = 4) = P(Z = 1) = p_Z(1)$
    - ➔ **This is incorrect!**
    - ➔ Clearly, $X \sim \mathcal{U}[0, 8]$, thus $p_X(4) = 1/8$
  - **Note:** $p_X(4) \neq p_Z(1)$
    - ✓ Due to the transformations of variables, we need to **scale densities** to **preserve the volume of probability mass**

Review: Change of Variable Formula

- **Change of Variables in 1-D**
  - **Change of Variable Formula (1-D):** If $X = f(Z)$ and $f(\cdot)$ is monotone with inverse $Z = f^{-1}(X) = h(X)$, then

$$p_X(x) = p_Z\big(h(x)\big) \cdot |h'(x)|$$

  - **Proof:** The corresponding infinitesimal intervals should have the same probability mass

$$p_Z(z)|dz| = p_X(x)|dx|$$

$$\Rightarrow \quad p_X(x) = p_Z(z)\frac{|dz|}{|dx|} = p_Z\big(h(x)\big)\left|\frac{dz}{dx}\right| = p_Z\big(h(x)\big)\,|h'(x)|$$

➔ We adjust the **volume** of each dx around x because h changes it

Review: Change of Variable Formula

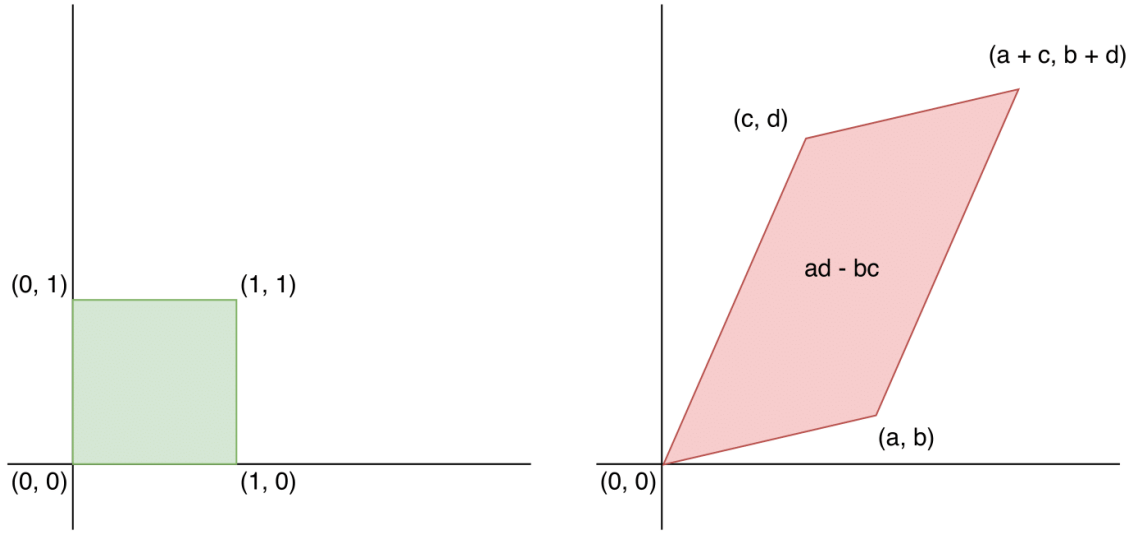
- **Change of Variables in 1-D**
  - **Change of Variable Formula (1-D):** If $X = f(Z)$ and $f(\cdot)$ is monotone with inverse $Z = f^{-1}(X) = h(X)$, then

$$p_X(x) = p_Z\big(h(x)\big) \cdot |h'(x)|$$

  - **Previous example:** If $X = 4Z$, $\mathrm{Z} \sim \mathcal{U}[0, 2]$, what is $p_X(4)$?
    - ✓ We have $f(Z) = 4Z$, $h(X) = X/4$
    - ✓ $p_X(4) = p_Z(1)|h'(4)| = \frac{1}{2} \times \left|\frac{1}{4}\right| = \frac{1}{8}$
  - ✓ **Note:** Change of variable formula, generalizes to higher dimensions via **determinants of transformations**

Review: Change of Variable Formula

- **Recap: Volume and Determinants (generalized to n-D)**
  - Let Z be a uniform random vector in $[0,1]^n$
  - Let $X = AZ$ for a square invertible matrix A, where $W = A^{-1}$



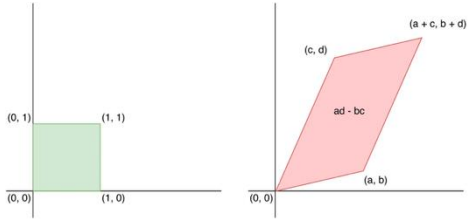
How is X distributed?

➔ Determinant $\det(A)$ still gives volume of the n-D shape

➔ X is still uniformly distributed over the parallelotope of area $|\det(A)|$

Review: Change of Variable Formula

- **Change of Variables Formula (Linear Case in n-D)**



  - Let Z be a uniform random vector in $[0,1]^n$
  - Let $X = AZ$ for a square invertible matrix A, where $W = A^{-1}$
  - **Change of Variables Formula:**

    X is uniformly distributed over the parallelotope of area $|\det(A)|$

$$p_X(\mathbf{x}) = p_Z(W\mathbf{x}) / |\det(A)|$$
$$= p_Z(W\mathbf{x})\,|\det(W)|$$

    where $\det(W) = \det(A^{-1}) = \frac{1}{\det(A)}$

  - **Note:** Similar to the 1-D case: $p_X(x) = p_Z\big(h(x)\big) \cdot |h'(x)|$

Review: Change of Variable Formula

- **Change of Variables Formula (General Case in n-D)**
  - **Linear Transformation A:** change of volume is given by $\det(A)$
  - **Non-linear Transformation $f(\cdot)$:** change of volume is given by the determinant of the Jacobian of $f(\cdot)$
  - Consider function $f: \mathbb{R}^n \to \mathbb{R}^m$
    - ✓ $x = (x_1, \dots, x_n)^\top,\ f(x) = \big(f_1(x), \dots, f_m(x)\big)^\top$
    - ✓ The Jacobian is defined as

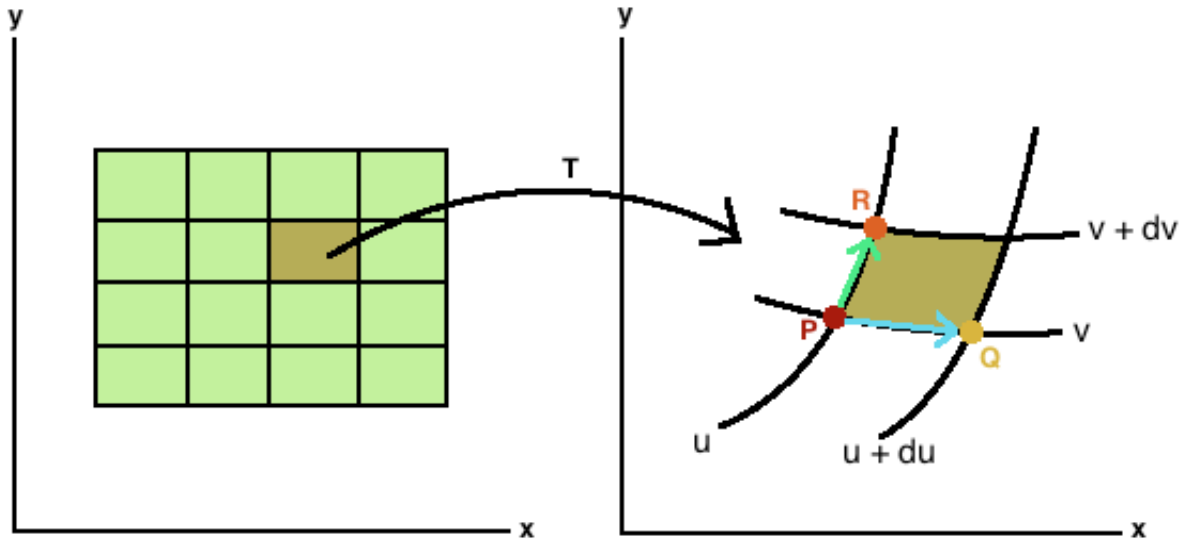
$$J = \frac{\partial \mathbf{f}}{\partial \mathbf{x}} = \begin{pmatrix} \frac{\partial f_1}{\partial x_1} & \cdots & \frac{\partial f_1}{\partial x_n} \\ \cdots & \cdots & \cdots \\ \frac{\partial f_m}{\partial x_1} & \cdots & \frac{\partial f_m}{\partial x_n} \end{pmatrix}$$

Generalization of gradient to multi-variate functions

Review: Change of Variable Formula

- **Change of Variables Formula (General Case in n-D)**
  - **Linear Transformation A:** change of volume is given by $\det(A)$
  - **Non-linear Transformation $f(\cdot)$:** change of volume is given by the determinant of the Jacobian of $f(\cdot)$

$dudv$



$\left|\det J_f(u,v)\right| dudv$

The mapping can be linearized for sufficiently small du, dv, i.e., the Jacobian J_f

Review: Change of Variable Formula

- **Change of Variables Formula (General Case in n-D)**
  - **Definition:** The mapping between Z and X, given by $f: \mathbb{R}^n \to \mathbb{R}^n$, is invertible such that $X = f(Z)$ and $Z = f^{-1}(X)$. We have

$$p_X(x) = p_Z\left(f^{-1}(x)\right)\left|\det\left(\frac{\partial f^{-1}(x)}{\partial x}\right)\right|$$

  - ✓ **Note ①:** Z and X are continuous and have the **same dimension**
  - ✓ **Note ②:** For any invertible matrix A, $\det(A^{-1}) = \det(A)^{-1}$. Therefore, we have

$$p_X(x) = p_Z(z)\left|det\left(\frac{\partial f(z)}{\partial z}\right)\right|^{-1}$$

$$\frac{\partial f^{-1}(x)}{\partial x} = \left(\frac{\partial f(z)}{\partial z}\right)^{-1}$$



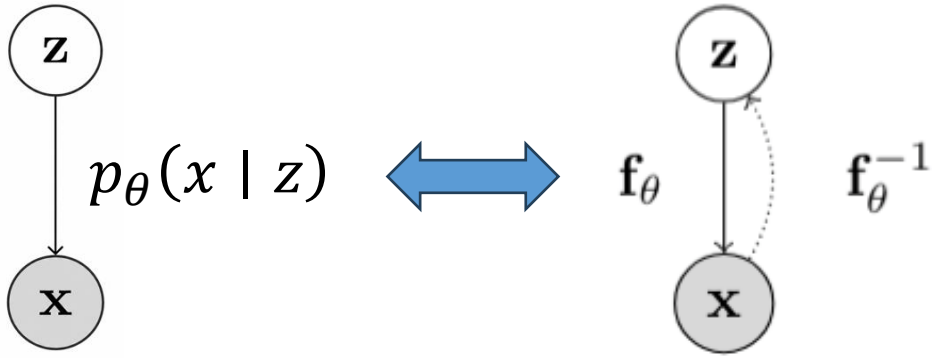
Lecture Outline

- Introduction to Flow-based Models
- Review: Change of Variable Formula
- Normalizing Flow Models
- Discussion and Advanced Topics

Normalizing Flow Models

- **Model Representation**
  - **Similar to VAE:** A normalizing flow model is a **latent variable model** over observed variables X and latent variables Z

Variational Autoencoder



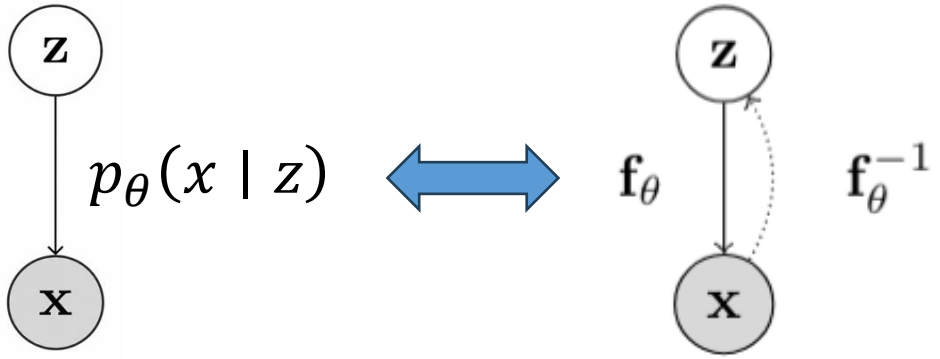
Normalizing Flow Models

  - **Unlike VAE:** The mapping function $f_\theta: \mathbb{R}^n \to \mathbb{R}^n$ between Z and X is **deterministic** and **invertible**, i.e, $X = f_\theta(Z)$ and $Z = f_\theta^{-1}(X)$
    - ✓ **VAE:** Using $p_\theta(x \mid z)$, its inversion (true posterior $p(z \mid x)$) is unknown

Normalizing Flow Models

- **Model Learning**

Variational Autoencoder



Normalizing Flow Models

➢ **Goal:** Learning $p_X(x;\theta)$ with the principle of **maximum likelihood**

✓ For VAE, the marginal likelihood $p_X(x;\theta)$ is generally intractable

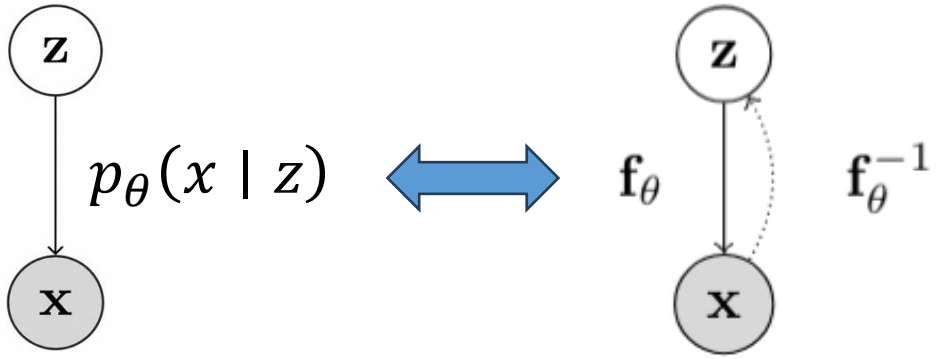
$$p_X(x;\theta) = \int p_\theta(x \mid z)\, p(z)\, dz$$

➔ **No deterministic one-to-one correspondence** between x and z, need to marginalizing over all latent explanations z

Normalizing Flow Models

- **Model Learning**

Variational Autoencoder



Normalizing Flow Models

➢ **Goal:** Learning $p_X(x;\theta)$ with the principle of **maximum likelihood**

✓ For Normalizing Flow models, $p_X(x;\theta)$ is given by

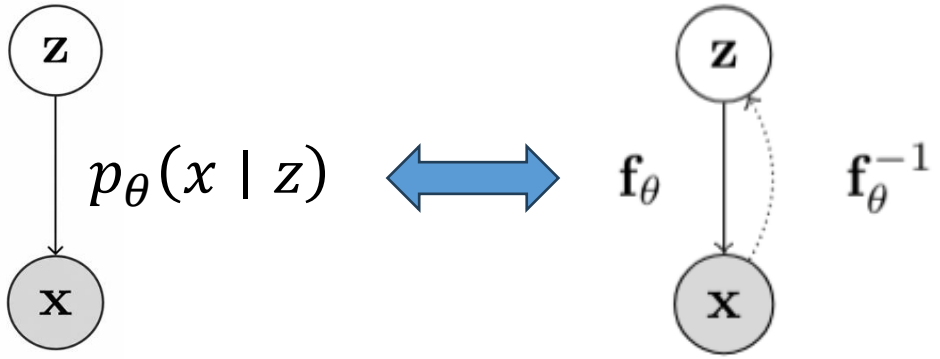
$$p_X(x;\theta) = p_Z\left(f_\theta^{-1}(x)\right)\left|\det\left(\frac{\partial f_\theta^{-1}(x)}{\partial x}\right)\right|$$

➔ Unlike VAE, the marginal likelihood can be **computed exactly!**

Normalizing Flow Models

- **Model Learning**

Variational Autoencoder



Normalizing Flow Models

➢ **Goal:** Learning $p_X(x;\theta)$ with the principle of **maximum likelihood**

✓ For Normalizing Flow models, $p_X(x;\theta)$ is given by

$$p_X(x;\theta) = p_Z\left(f_\theta^{-1}(x)\right)\left|\det\left(\frac{\partial f_\theta^{-1}(x)}{\partial x}\right)\right|$$

How to construct a **invertible** and **tractable** transformation function f_θ?

Normalizing Flow Models

- **How to Construct Transformation f?**
  - **General Solution: Compose** simple invertible mappings

    ① Start with a simple prior distribution for $\boldsymbol{z}_0$ (e.g., Gaussian)

    ② Apply **sequence of M simple invertible transformations** ($x \triangleq z_M$)

$$z_M \coloneqq f_\theta^M \circ \cdots \circ f_\theta^1(z_0) = f_\theta^M\left(f_\theta^{M-1}\left(\cdots\left(f_\theta^1(z_0)\right)\right)\right) \triangleq f_\theta(z_0)$$

    ③ Using **change of variables formula**

$$p_X(x;\theta) = p_Z\left(f_\theta^{-1}(x)\right)\prod_{m=1}^{M}\left|\det\left(\frac{\partial(f_\theta^m)^{-1}(z_m)}{\partial z_m}\right)\right|$$

    (The determinant of composition equals product of determinants)

Normalizing Flow Models

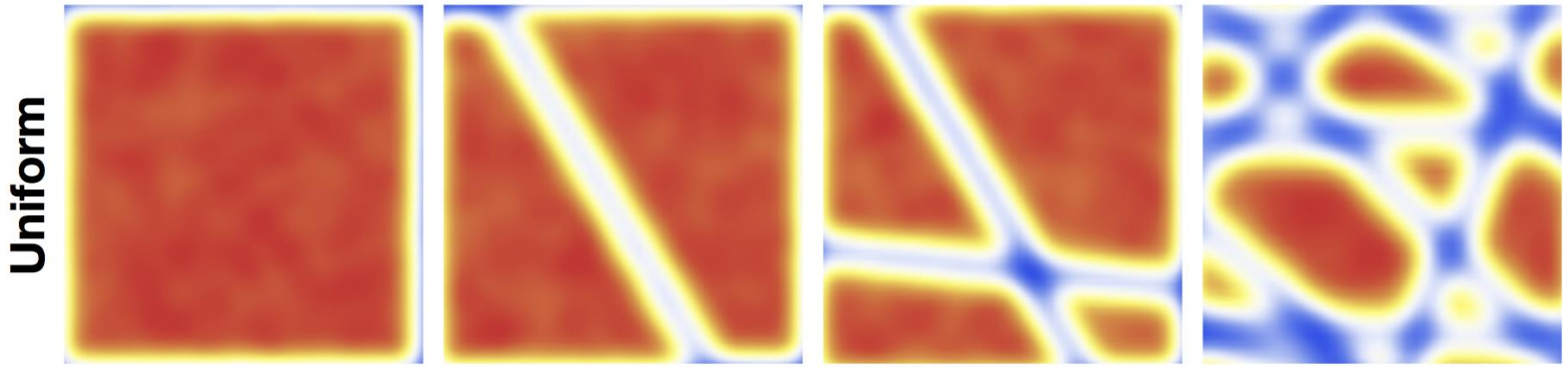
- **A Toy Example: Planar Flows**
  - **Invertible Transformation**

$$x = f_\theta(z) := z + u \cdot h(w^\top z + b)$$

Warp the distribution relative to a hyperplane

parameterized by $\theta = (w, u, b)$, and $h(\cdot)$ provides non-linearity

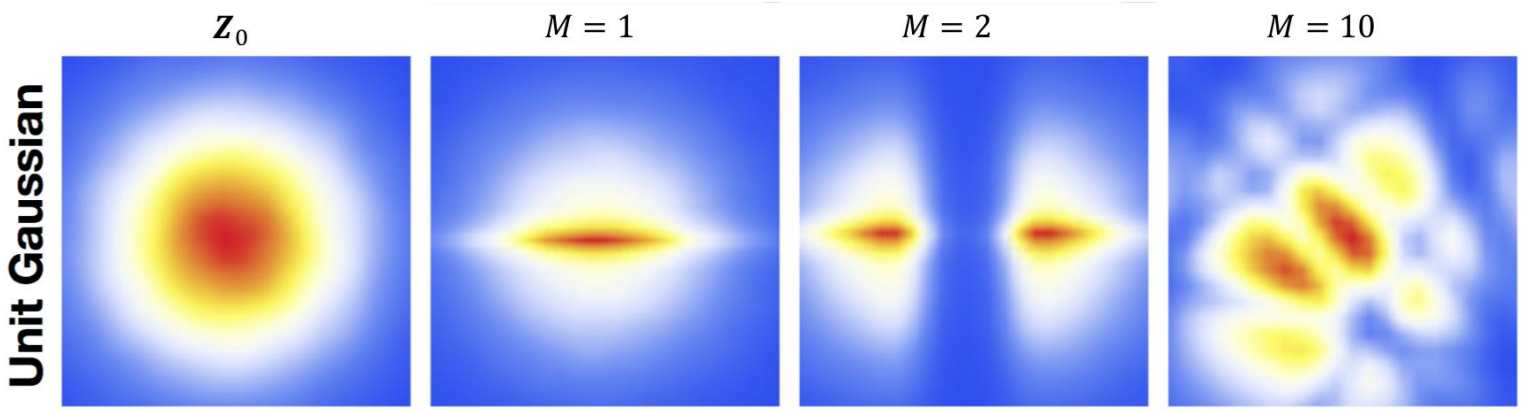
  - **Visualization of Transformation**



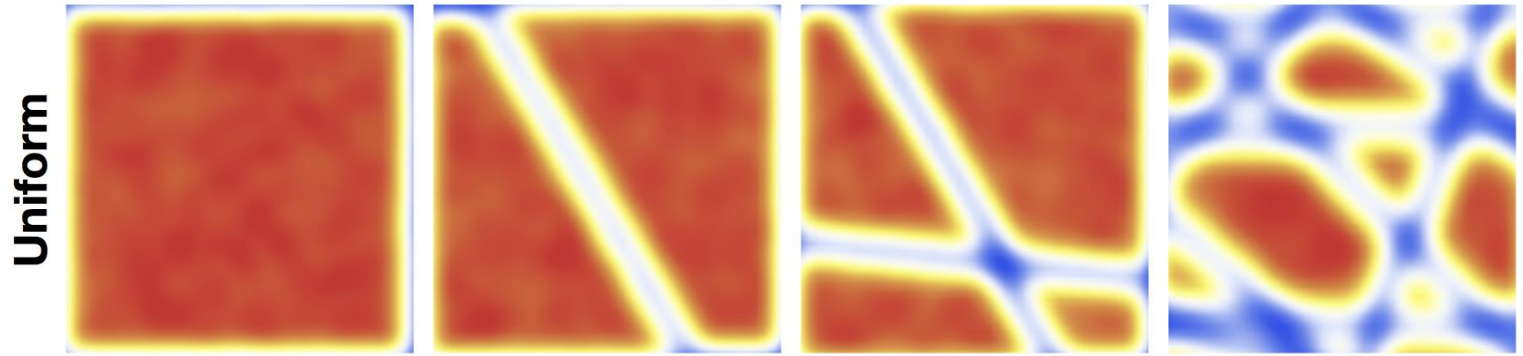
Transformation after 0, 1, 2, ..., 10 recursive applications

Normalizing Flow Models

- **A Toy Example: Planar Flows**
  - Base distribution: Gaussian



  - Base distribution: Uniform



Complex Distributions by 10 transformations



Lecture Outline

- Introduction to Flow-based Models
- Review: Change of Variable Formula
- Normalizing Flow Models
- Discussion and Advanced Topics

Discussion and Advanced Topics

- **Learning and Inference**
  - **Learning:** Maximum likelihood over the dataset $\mathcal{D}$

$$\max_{\theta} \log p_X(\mathcal{D};\theta) = \max_{\theta} \sum_{x\in\mathcal{D}} \left[\log p_Z\left(f_\theta^{-1}(x)\right) + \log \left| \det\left(\frac{\partial f_\theta^{-1}(x)}{\partial x}\right)\right|\right]$$

    - ✓ **Note: Likelihood can be evaluated exactly**, due to invertible transforms and change of variable formula
  - **Sampling:** By direct **forward transformation**

$$z \sim p_Z(z), x = f_\theta(z)$$

  - **Inference:** Latent representations inferred via **inverse transformation**

$$z = f_\theta^{-1}(x)$$

Discussion and Advanced Topics

- **How to Design a Normalizing Flow Model?**
  - **Simple Prior** $p_Z(z)$: allows for efficient sampling
  - **Invertible Transformation** f_θ:
    - ✓ Likelihood computation: efficient evaluation of the mapping $x \to z$
    - ✓ Sampling: efficient evaluation of the mapping $z \to x$
  - **Key Challenge**:

$$p_X(x;\theta) = p_Z\left(f_\theta^{-1}(x)\right)\left|\det\left(\frac{\partial f_\theta^{-1}(x)}{\partial x}\right)\right|$$

Determinants of $n \times n$ Jacobian matrices is required to compute the likelihood (n is data dimensionality) ➔ **Complexity: $O(n^3)$, very expensive!**

Discussion and Advanced Topics

- **How to Compute Determinants Efficiently?**
  - ➢ **Motivation:** Design f_θ so that the Jacobian has special structure
    - ✓ Determinant of a **triangular matrix** is the **product of the diagonal entries**, and thus an $\boldsymbol{O(n)}$ operation

$$\mathbf{x} = (x_1, \cdots, x_n) = \mathbf{f}(\mathbf{z}) = (f_1(\mathbf{z}), \cdots, f_n(\mathbf{z}))$$

$$J = \frac{\partial \mathbf{f}}{\partial \mathbf{z}} = \begin{pmatrix} \frac{\partial f_1}{\partial z_1} & \cdots & \frac{\partial f_1}{\partial z_n} \\ \cdots & \cdots & \cdots \\ \frac{\partial f_n}{\partial z_1} & \cdots & \frac{\partial f_n}{\partial z_n} \end{pmatrix} \xrightarrow{x_i = f_i(\mathbf{z})} J = \frac{\partial \mathbf{f}}{\partial \mathbf{z}} = \begin{pmatrix} \frac{\partial f_1}{\partial z_1} & \cdots & 0 \\ \cdots & \cdots & \cdots \\ \frac{\partial f_n}{\partial z_1} & \cdots & \frac{\partial f_n}{\partial z_n} \end{pmatrix}$$

Only depends on $\boldsymbol{z_{<i}}$

$\boldsymbol{O(n^3)}$ $\qquad$ $\boldsymbol{O(n)}$

  - ➢ **Goal:** Design f_θ with triangular Jacobian matrix ➔ Linear complexity

Summary

- **Motivation:** Create a latent variable model with exact likelihood evaluation
- **Normalizing Flow:**

  Create a deterministic mapping f_θ between x and z. For a distribution $p_Z(z)$

$$p_X(x) = p_Z(z) \left| det \left(\frac{\partial f(z)}{\partial z} \right) \right|^{-1}$$

- **Challenge:** Computing the determinant of Jacobian matrix is expensive
- **Solution:** Design f_θ with Jacobian matrix of special structure

$$J = \frac{\partial \mathbf{f}}{\partial \mathbf{z}} = \begin{pmatrix} \frac{\partial f_1}{\partial z_1} & \cdots & \frac{\partial f_1}{\partial z_n} \\ \cdots & \cdots & \cdots \\ \frac{\partial f_n}{\partial z_1} & \cdots & \frac{\partial f_n}{\partial z_n} \end{pmatrix} \xrightarrow{x_i = f_i(\mathbf{z})} J = \frac{\partial \mathbf{f}}{\partial \mathbf{z}} = \begin{pmatrix} \frac{\partial f_1}{\partial z_1} & \cdots & 0 \\ \cdots & \cdots & \cdots \\ \frac{\partial f_n}{\partial z_1} & \cdots & \frac{\partial f_n}{\partial z_n} \end{pmatrix}$$

Only depends on $z_{<i}$

$O(n^3)$ $O(n)$

Supplementary Material

- **Stanford CS 236 Lecture 7**
  - https://www.youtube.com/watch?v=m6dKKRsZwBQ&list=PLoROMvodv4rPOWA-omMM6STXaWW4FvJT8&index=7
- **Cornell CS 6785 Lecture 7**
  - https://www.youtube.com/watch?v=3lqRV40LKPs&list=PL2UML_KCiC0UPzjW9BjO-lW6dqliu9O4B&index=7
- **Deep Generative Models**
  - Chapter 4 Flow-based Models

Supplementary Material

- **Reading List**
  - D. Rezende and S. Mohamed, Variational Inference with Normalizing Flows, ICML 2015
  - D. P. Kingma and P. Dhariwal, Natural Images, Glow: Generative Flow with Invertible 1x1 Convolutions, NeurIPS 2018
  - J. Behrmann et al., Invertible residual networks, ICML 2019
- **Blog Post**
  - Flow-based Deep Generative Models (https://lilianweng.github.io/posts/2018-10-13-flow-models/)